Stage 1: RTL-to-MLIR Extraction

Synthesizable Verilog Files
(Flattened Netlist)

Instruction Harness
(Instruction Names, Opcode / Funct encoding)

Modified autoGenILA

(Two-Pass: Taint Analysis & Symbolic Unrolling over Cycle Bound)

Taint-based ASV Inference

Constraint DAG Traversal (Arith, Mux, Bit-select, etc.)

New MLIR UpdateFunctionGen (Replacing LLVM iR)

TensorLift Modification:
[1] MLIR Backend (arith, memref, scf, affine dialects)
[2] Time-indexed Input Memrefs (Grouped time series)
[3] Automation Harness (ASV Discovery, Encoding Discovery)

Per-(Instruction, ASV) MLIR functions
(func.func in arith, memref, scf, affine)

Stage 2: Semantic Lifting

tensorlift-opt tool: Eight-Pass MLIR Rewrite Pipeline

Pass descriptions detailed in Table 2

[A] Bit-manip Canonicalization:

A1 canon-bitmanip (Collapses sign-extension chains to arith.extsi)
A2 narrow-types

[B] Idiom Detection:

B3 detect-mac (Walks addi to find multiplier, tags with tensorlift.mac)
B4 specialize-control (Constant-folds fixed control, eliminates select chains)
B5 detect-clamp (Recognized saturation idiom ext(trunci(x)))

[C] Loop/Tensor-Shape Reconstruction:

C6 reconstruct-loops (Builds def-use chains over MAC, materialized scf.for reduction)
C7 lift-to-linalg (Lifts scf.for reduction to linalg dot-product)

[D] TAIDL Metadata Emission:

D8 emit-taidl-metadata (Annotates with taidl.* attributes, classifies arguments, infers dimensions)

Tensor-Level Structured MLIR
(annotated with taidl.*)

Stage 3: TAIDL Assembly

Annotated MLIR
from Stage2

Python Assembler
(Materializes TAIDL Specification)

CISC Macro Composition
(canonical HLO body (matmul or conv) over config-staged state bounds)

Compute Path
(HLO templatess: mac/dot_product -> dot+add pool -> reduce(max))

DMA Path
(Driven by taidl.dma_role tags, emits load or store)

Inferring the Finite State Machine
FSM register scoring
State-update recovery
(from LLVM IR or RTL)

ACT-Consumable TAIDL Specification
(XLA-HLO Style semantics)

Hand-Written Reference

VS.

Inferred TAIDL